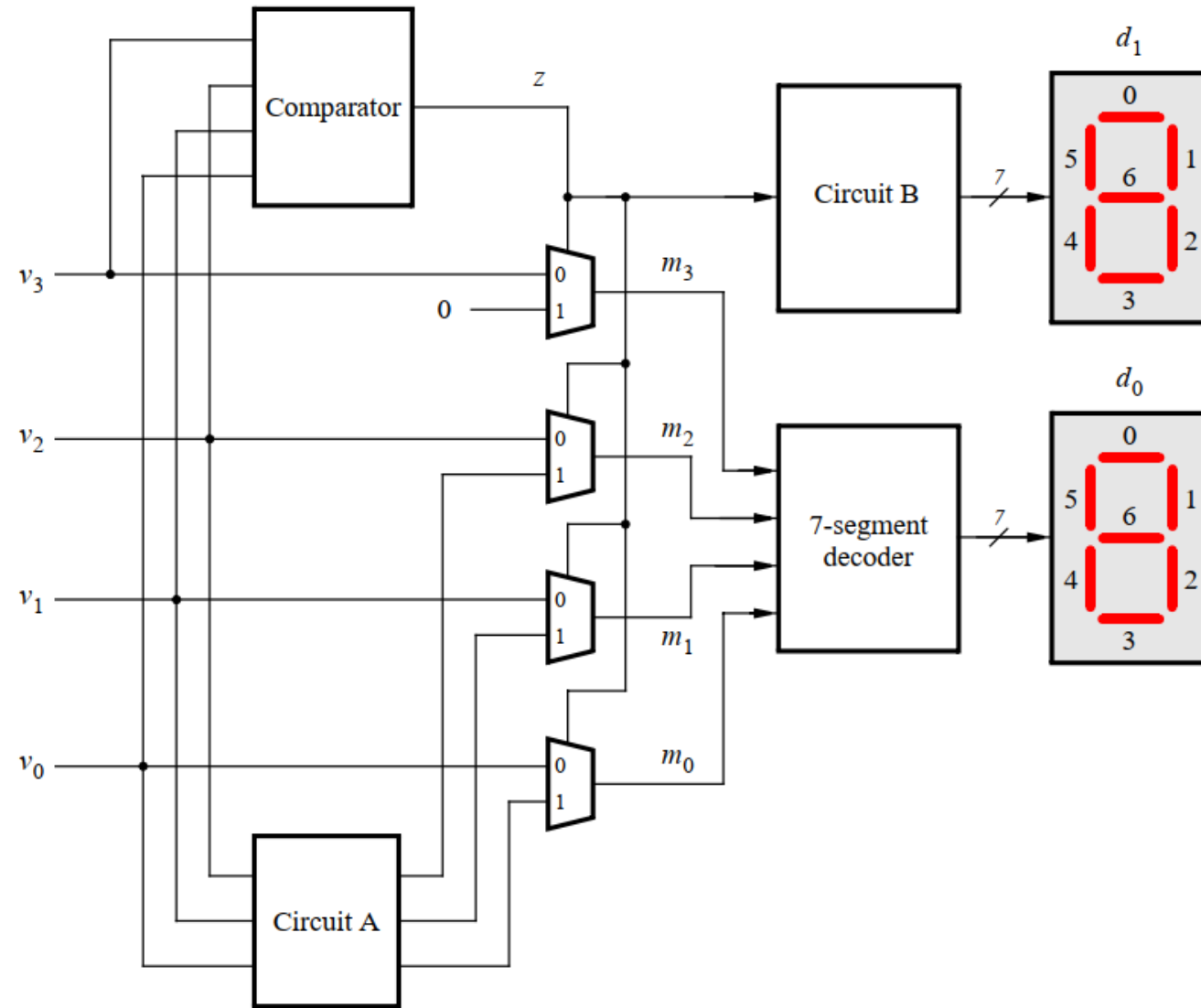
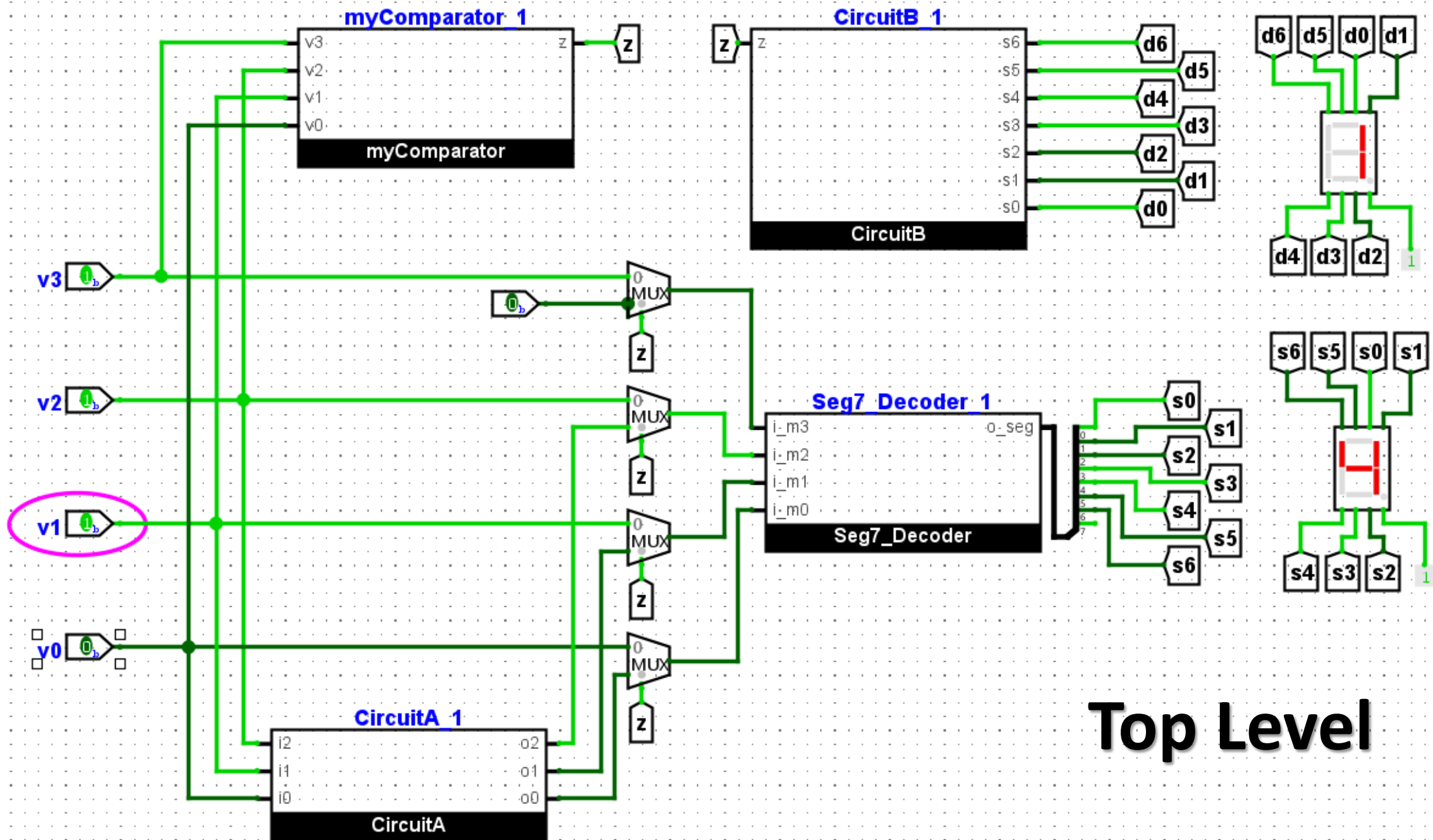


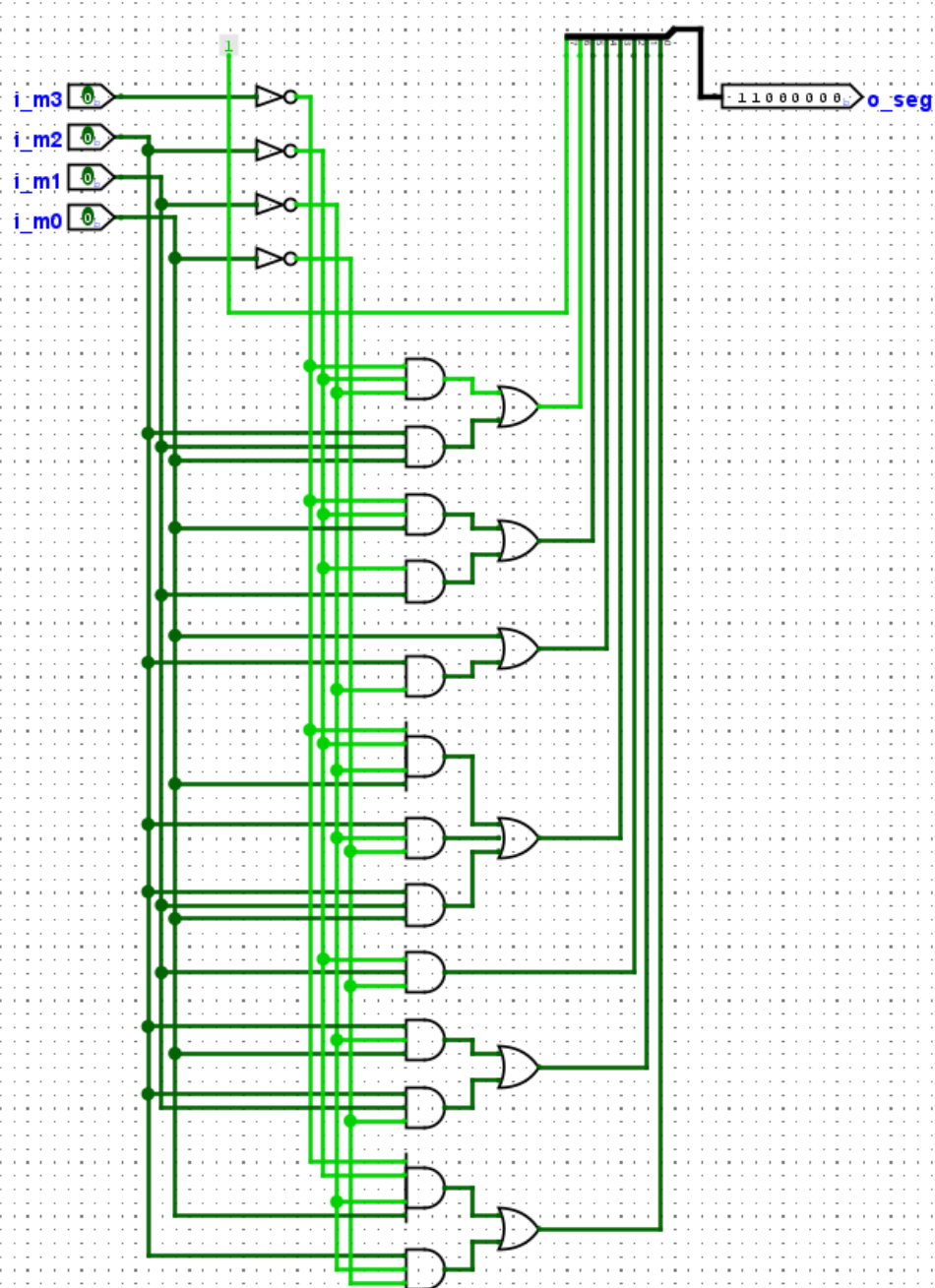
# Part II





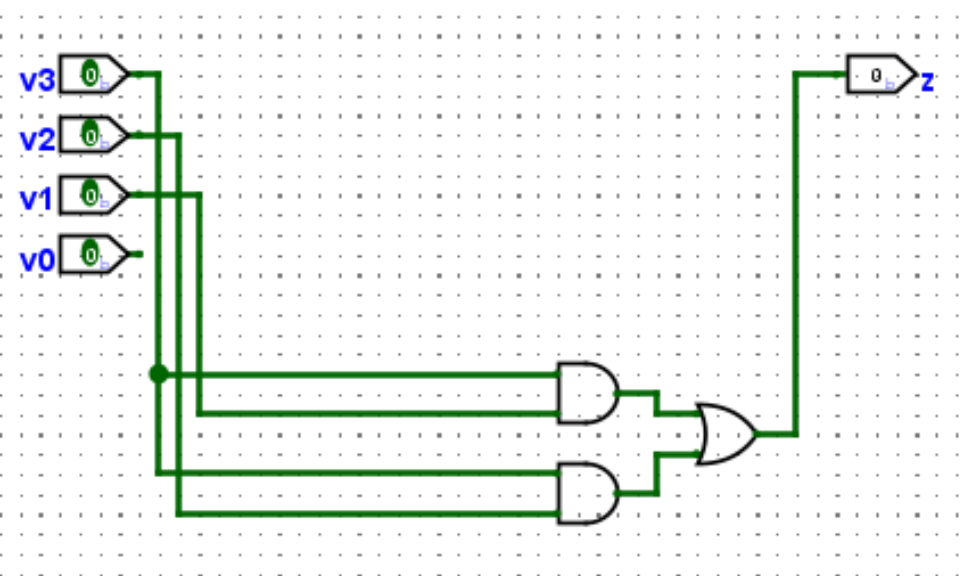
Top Level

# 7 Segment Decoder



## Input output Relation

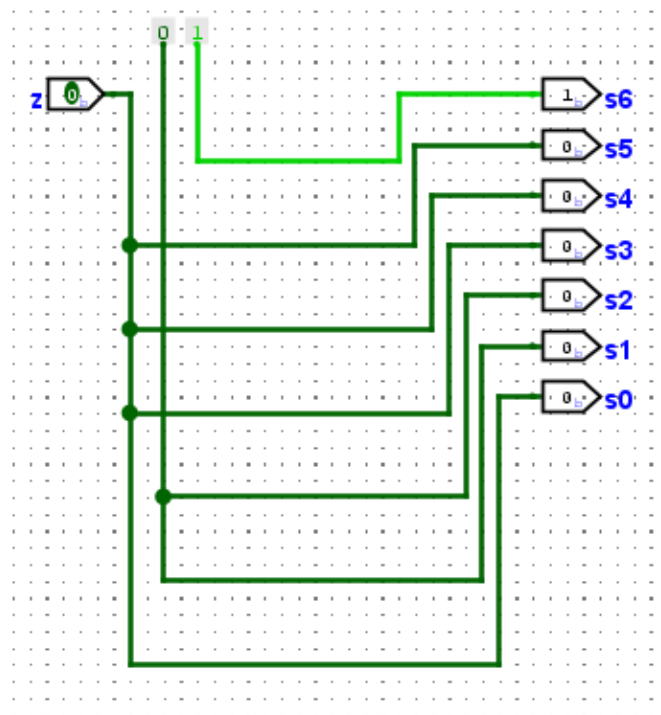
$$\begin{aligned}
 o_{seg_7} &= 1 \\
 o_{seg_6} &= \overline{i_{m3} \cdot i_{m2} \cdot i_{m1}} + \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}} \\
 o_{seg_5} &= \overline{i_{m3} \cdot i_{m2} \cdot i_{m0}} + \overline{i_{m2} \cdot i_{m1}} \\
 o_{seg_4} &= i_{m0} + \overline{i_{m2} \cdot i_{m1}} \\
 o_{seg_3} &= \overline{i_{m3} \cdot i_{m2} \cdot i_{m1} \cdot i_{m0}} + \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}} + \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}} \\
 o_{seg_2} &= \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}} \\
 o_{seg_1} &= \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}} + \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}} \\
 o_{seg_0} &= \overline{i_{m3} \cdot i_{m2} \cdot i_{m1} \cdot i_{m0}} + \overline{i_{m2} \cdot i_{m1} \cdot i_{m0}}
 \end{aligned}$$



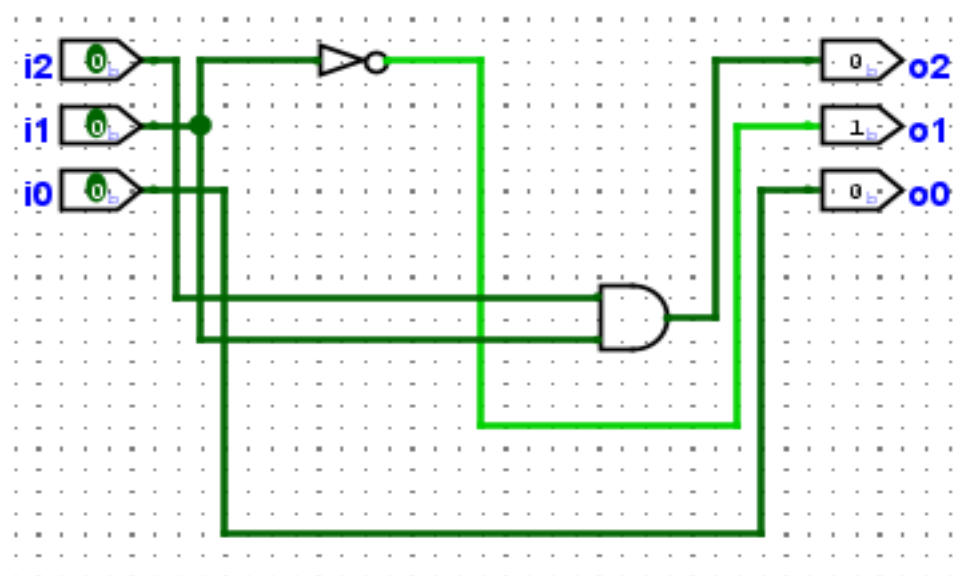
$$z = v3 \cdot v1 + v3 \cdot v2$$

Comparator

Circuit B



$$\begin{aligned} s6 &= 1 \\ s5 &= z \\ s4 &= z \\ s3 &= z \\ s2 &= 0 \\ s1 &= 0 \\ s0 &= z \end{aligned}$$



$$\begin{aligned} o2 &= i2 \cdot i1 \\ o1 &= \overline{i1} \\ o0 &= i0 \end{aligned}$$

Circuit A

# Verilog Code Top Module

```
module main(  
    input [9:0] SW,  
    output [9:0] LEDR,  
  
    input      MAX10_CLK1_50,  
    output [7:0] HEX0,  
    output [7:0] HEX1,  
    output [7:0] HEX2,  
    output [7:0] HEX3,  
    output [7:0] HEX4,  
    output [7:0] HEX5  
);  
  
    // PART II  
  
    wire [3:0] w_m;  
  
    // 7 Segment Display  
    Seg7_Decoder D0(  
        .m(w_m), // 4bits  
        .out(HEX0) // 8bits  
    );
```

1

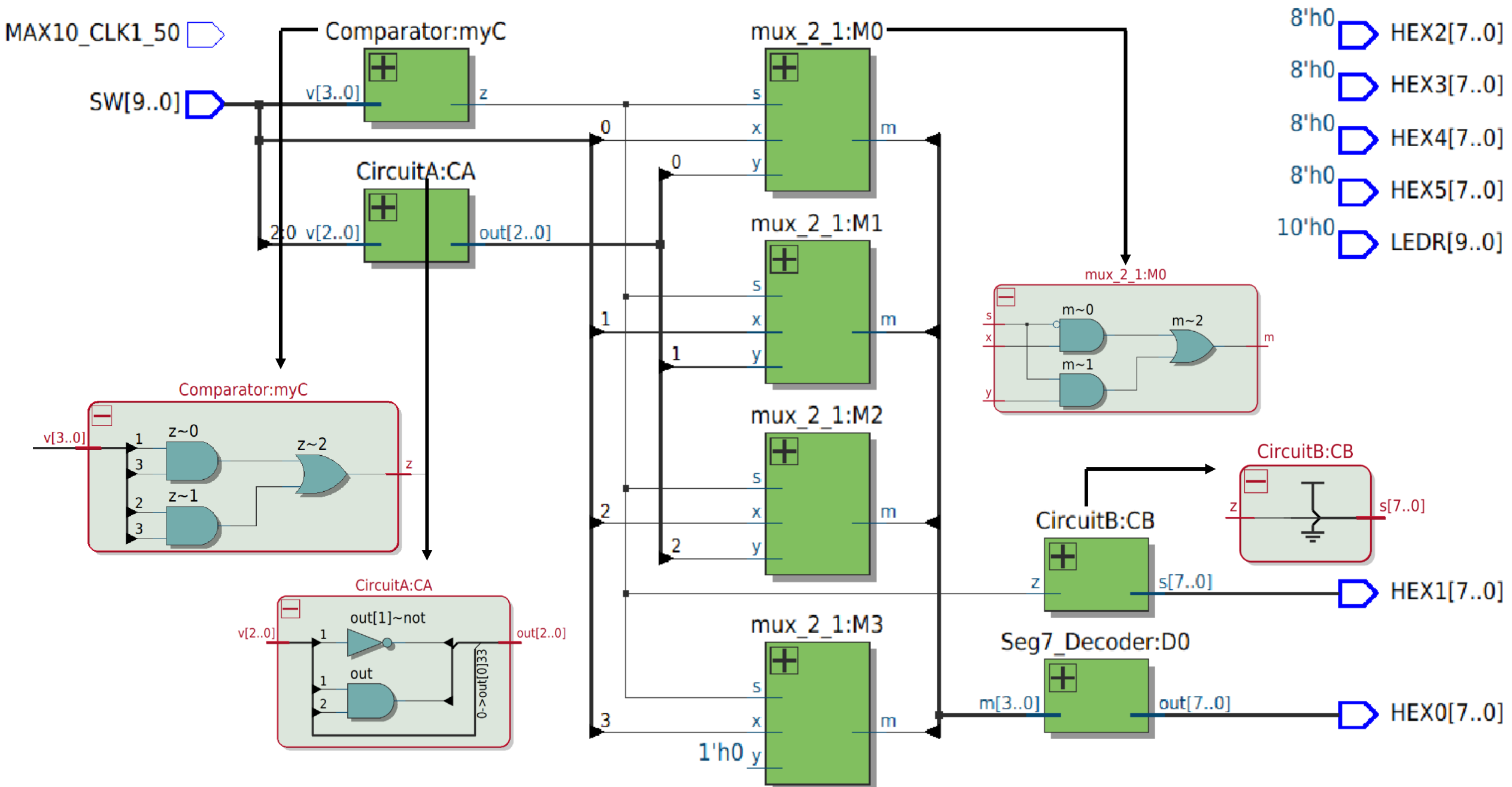
```
    wire w_z;  
  
    wire [2:0] w_ca;  
    // Multiplexers  
    mux_2_1 M0(  
        .s(w_z),  
        .x(SW[0]),  
        .y(w_ca[0]),  
        .m(w_m[0])  
    );  
  
    mux_2_1 M1(  
        .s(w_z),  
        .x(SW[1]),  
        .y(w_ca[1]),  
        .m(w_m[1])  
    );  
  
    mux_2_1 M2(  
        .s(w_z),  
        .x(SW[2]),  
        .y(w_ca[2]),  
        .m(w_m[2])  
    );
```

2

```
    mux_2_1 M3(  
        .s(w_z),  
        .x(SW[3]),  
        .y(1'b0),  
        .m(w_m[3])  
    );  
  
    // Circuit A  
    CircuitA CA(  
        .v({SW[2], SW[1], SW[0]}),  
        .out(w_ca)  
    );  
  
    // Comparator  
    Comparator myC(  
        .v({SW[3], SW[2], SW[1], SW[0]}),  
        .z(w_z)  
    );  
  
    // Circuit B  
    CircuitB CB(  
        .z(w_z),  
        .s(HEX1)  
    );  
  
endmodule
```

3

## RTL Schematic TOP LEVEL



## Comparator

```
module CircuitA(  
    input [2:0] v,  
    output [2:0] out  
);  
  
    assign out[2] = v[2]&v[1];  
    assign out[1] = ~v[1];  
    assign out[0] = v[0];  
  
endmodule
```

## Circuit A

**2 to 1 Mux**  
**1 bit**

```
module Comparator(  
    input [3:0] v,  
    output z  
);  
  
    assign z = v[3]&v[1]|v[3]&v[2];  
  
endmodule
```

```
module mux_2_1(  
    input s,  
    input x,  
    input y,  
    output m  
);  
  
    assign m = (~s&x)|(s&y);  
  
endmodule
```

## Circuit B

```
module CircuitB(  
    input z,  
    output [7:0] s  
);  
  
    assign s[7] = 1'b1;  
    assign s[6] = 1'b1;  
    assign s[5] = z;  
    assign s[4] = z;  
    assign s[3] = z;  
    assign s[2] = 1'b0;  
    assign s[1] = 1'b0;  
    assign s[0] = z;  
  
endmodule
```

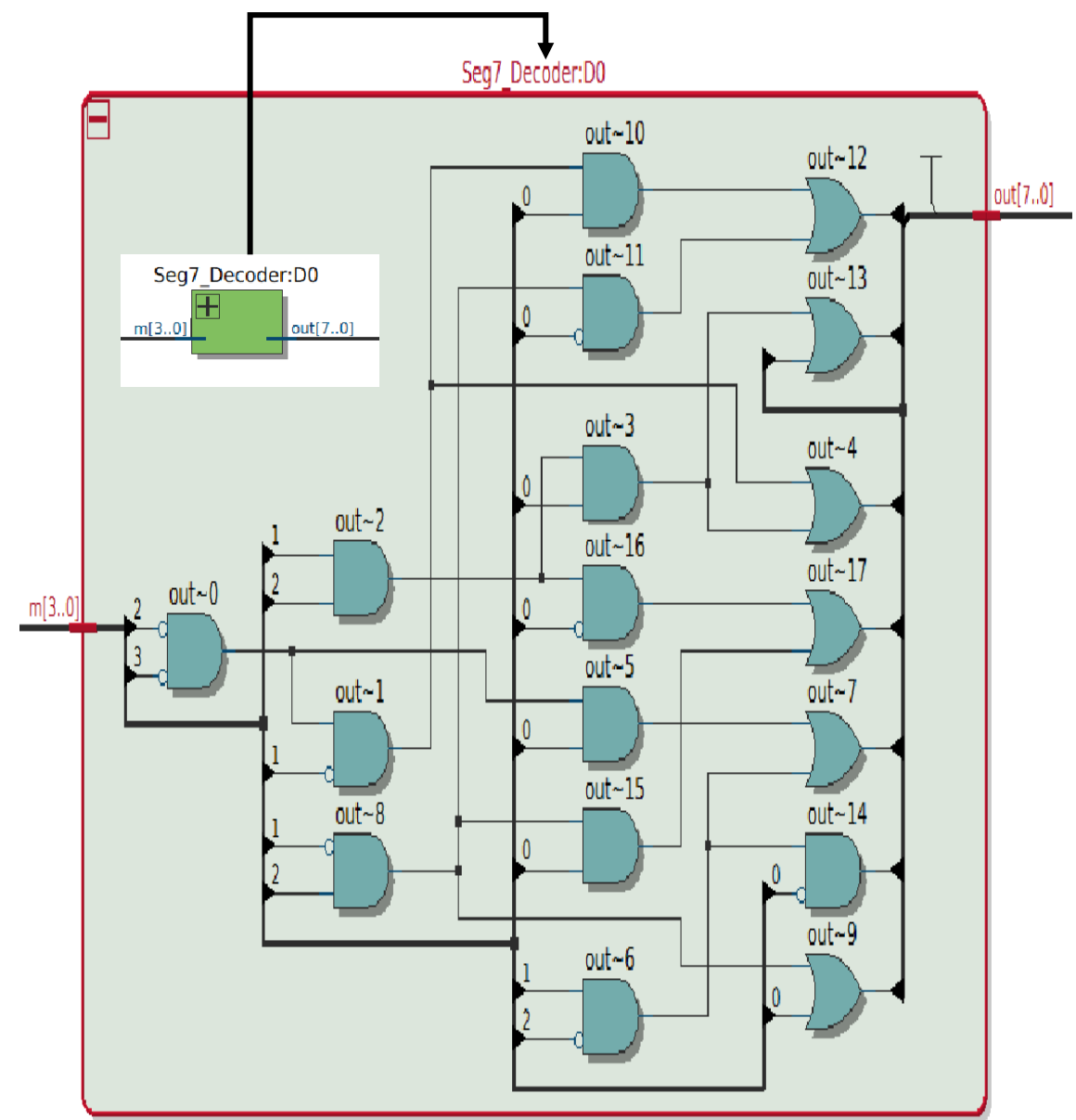
```

module Seg7_Decoder(
    input [3:0] m,
    output [7:0] out
);

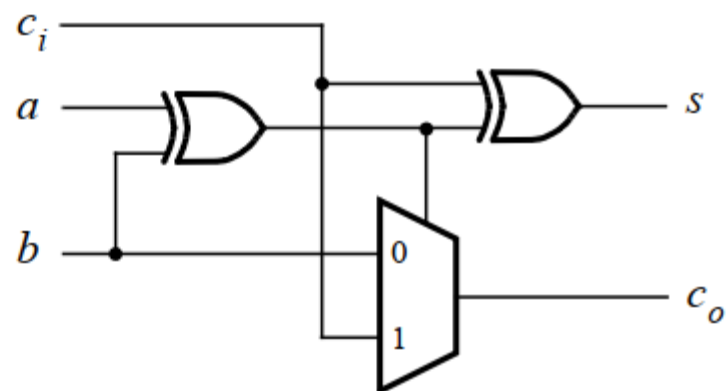
    //
    assign out[7] = 1'b1;
    assign out[6] =
        ~m[3]&~m[2]&~m[1]|m[2]&m[1]&m[0];
    assign out[5] =
        ~m[3]&~m[2]&m[0]|~m[2]&m[1];
    assign out[4] = m[0]|m[2]&~m[1];
    assign out[3] =
        ~m[3]&~m[2]&~m[1]&m[0]|m[2]&~m[1]&~m[0]|m[2]&
        m[1]&m[0];
    assign out[2] = ~m[2]&m[1]&~m[0];
    assign out[1] =
        m[2]&~m[1]&m[0]|m[2]&m[1]&~m[0];
    assign out[0] =
        ~m[3]&~m[2]&~m[1]&m[0]|m[2]&~m[1]&~m[0];

endmodule

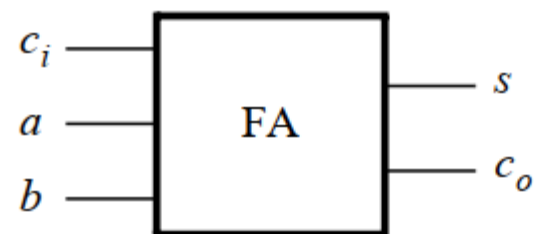
```



**Segment 7  
Decoder**



a) Full adder circuit

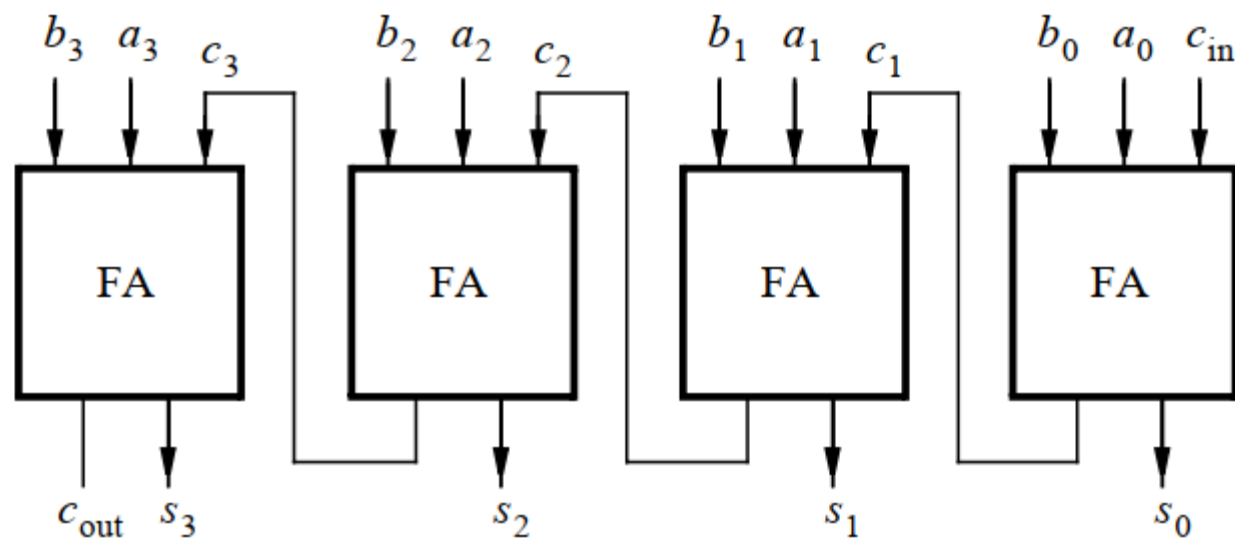


b) Full adder symbol

## Part III

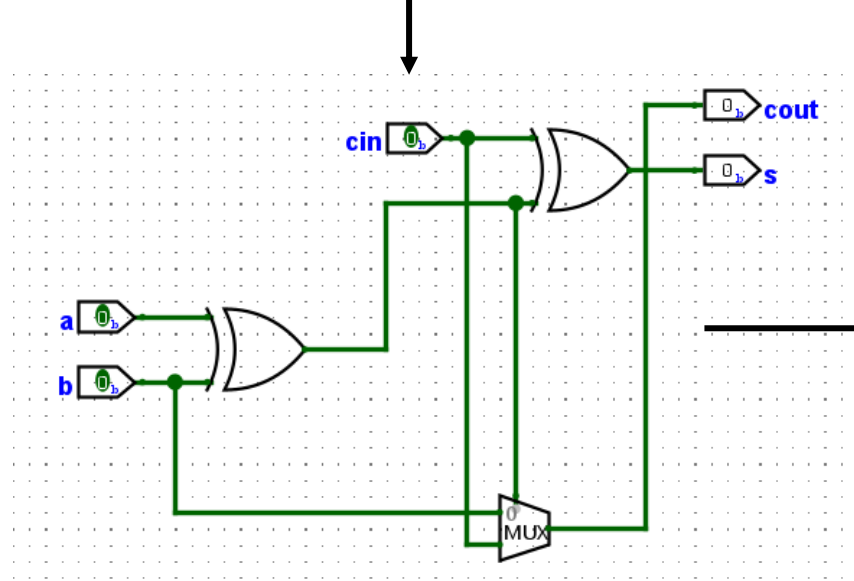
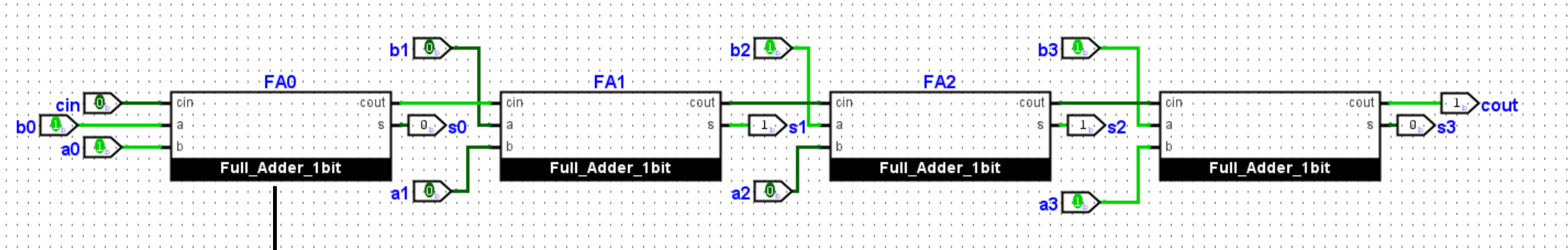
| $b$ | $a$ | $c_i$ | $c_o \ s$ |   |
|-----|-----|-------|-----------|---|
| 0   | 0   | 0     | 0         | 0 |
| 0   | 0   | 1     | 0         | 1 |
| 0   | 1   | 0     | 0         | 1 |
| 0   | 1   | 1     | 1         | 0 |
| 1   | 0   | 0     | 0         | 1 |
| 1   | 0   | 1     | 1         | 0 |
| 1   | 1   | 0     | 1         | 0 |
| 1   | 1   | 1     | 1         | 1 |

c) Full adder truth table



d) Four-bit ripple-carry adder circuit

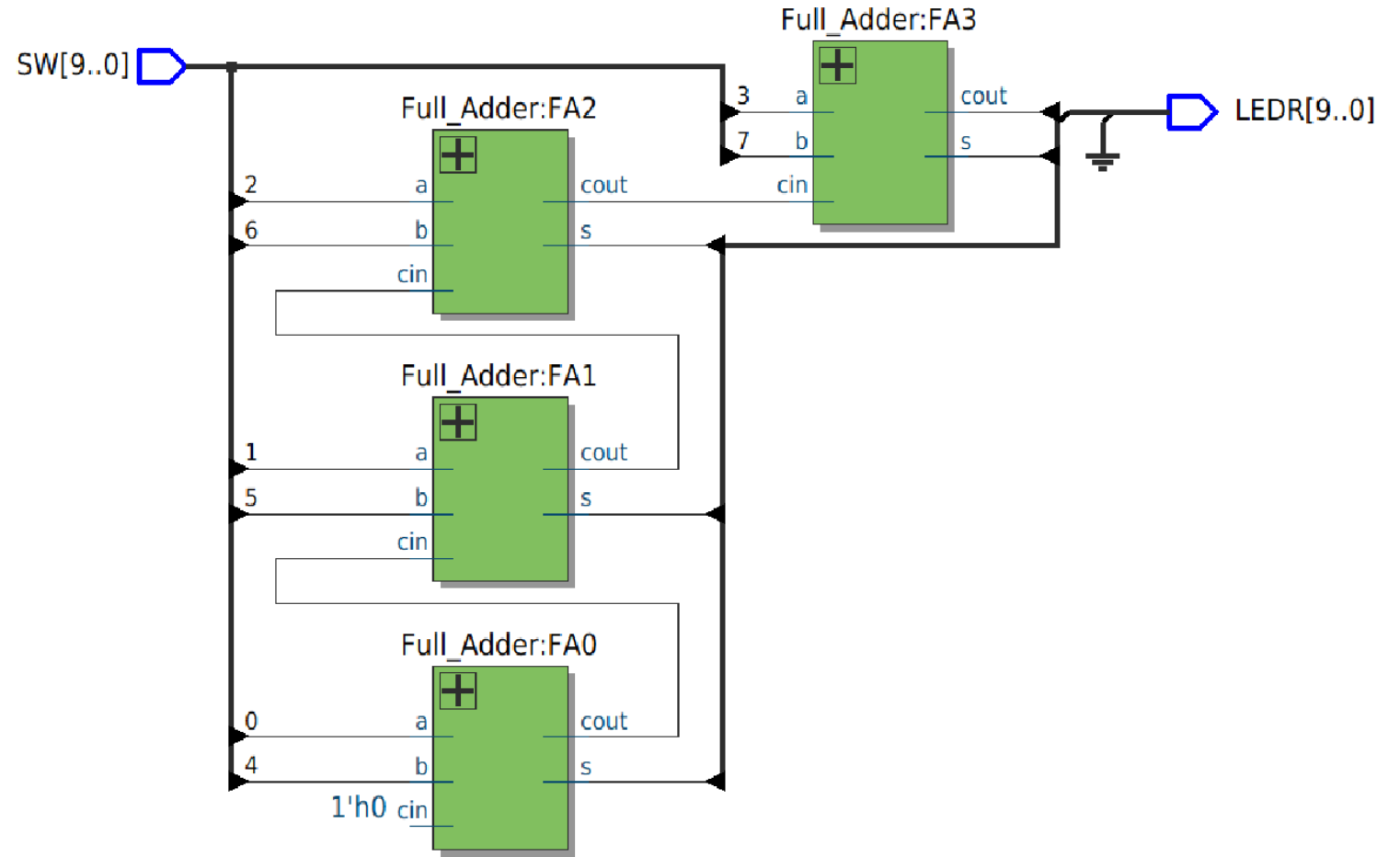
# Top Level



| $c_{in}$ | $a$ | $b$ | $s$ | $c_{out}$ |
|----------|-----|-----|-----|-----------|
| 0        | 0   | 0   | 0   | 0         |
| 0        | 0   | 1   | 1   | 0         |
| 0        | 1   | 0   | 1   | 0         |
| 0        | 1   | 1   | 0   | 1         |
| 1        | 0   | 0   | 1   | 0         |
| 1        | 0   | 1   | 0   | 1         |
| 1        | 1   | 0   | 0   | 1         |
| 1        | 1   | 1   | 1   | 1         |

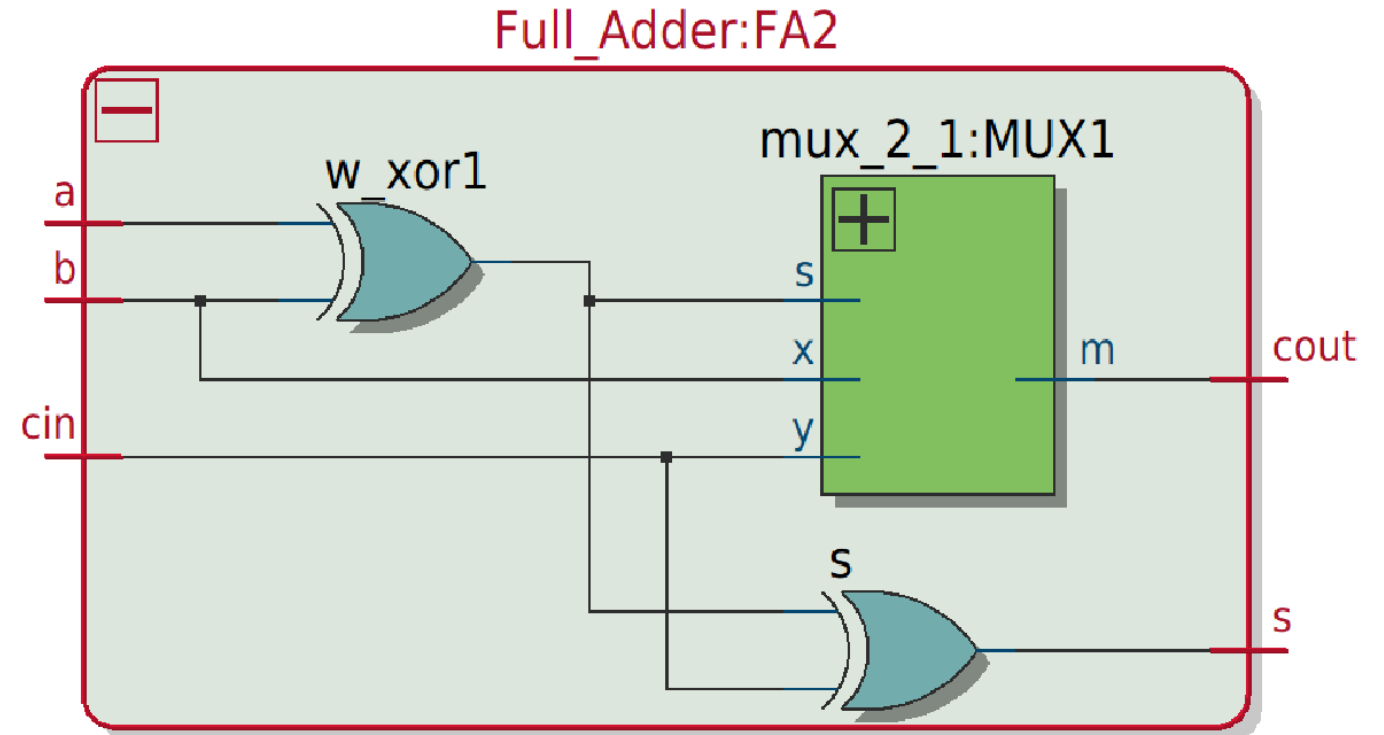
## Main module

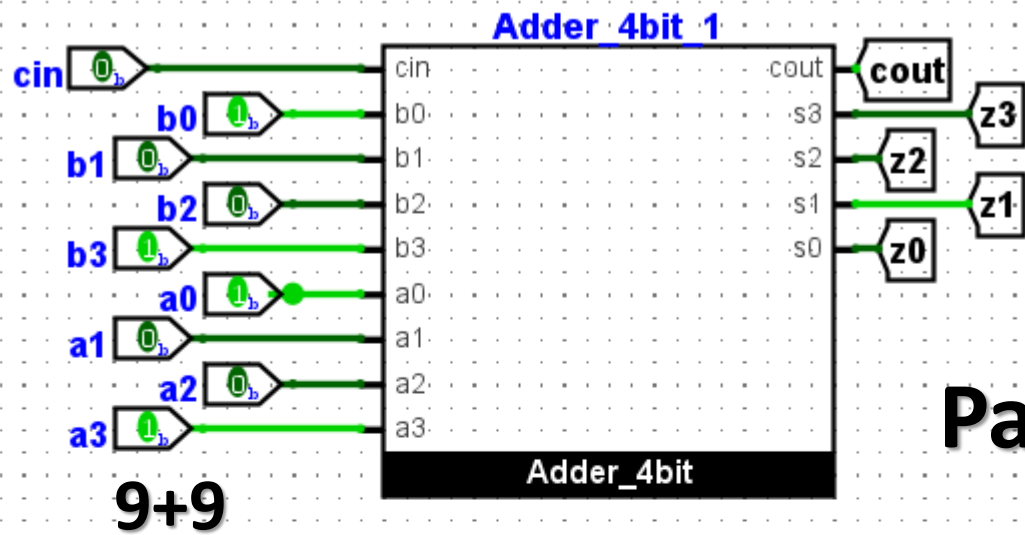
```
module main(  
    // pinout assignment  
    input      [9:0] SW,  
    output     [9:0] LEDR  
);  
  
    wire c0, c1, c2;  
  
    Full_Adder FA0(  
        .a(SW[0]), .b(SW[4]), .cin(1'b0),  
        .s(LEDR[0]), .cout(c0)  
    );  
  
    Full_Adder FA1(  
        .a(SW[1]), .b(SW[5]), .cin(c0),  
        .s(LEDR[1]), .cout(c1)  
    );  
  
    Full_Adder FA2(  
        .a(SW[2]), .b(SW[6]), .cin(c1),  
        .s(LEDR[2]), .cout(c2)  
    );  
  
    Full_Adder FA3(  
        .a(SW[3]), .b(SW[7]), .cin(c2),  
        .s(LEDR[3]), .cout(LED9)  
    );  
  
endmodule
```



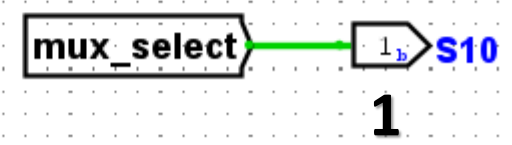
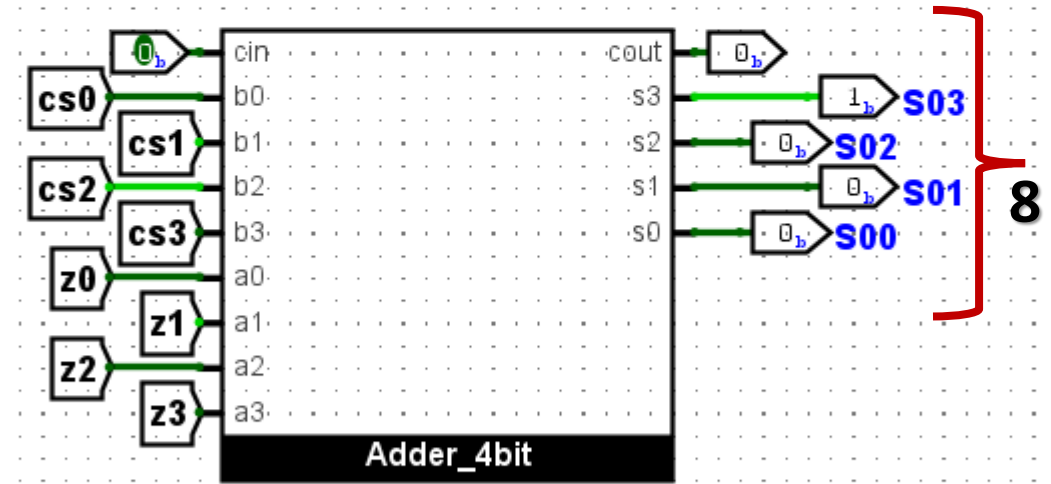
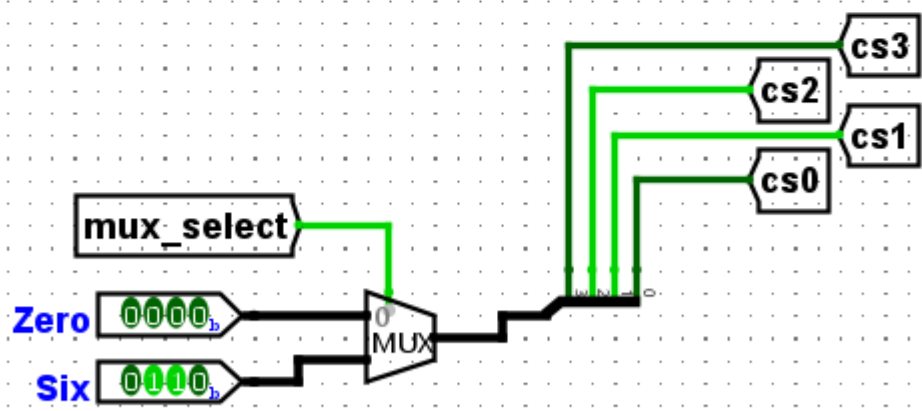
## Full Adder Module

```
module Full_Adder(  
    input a, b, cin,  
    output s, cout  
);  
  
    wire w_xor1;  
  
    assign w_xor1 = a ^ b;  
    assign s = w_xor1 ^ cin;  
  
    mux_2_1 MUX1(  
        .s(w_xor1),  
        .x(b),  
        .y(cin),  
        .m(cout)  
    );  
  
endmodule
```

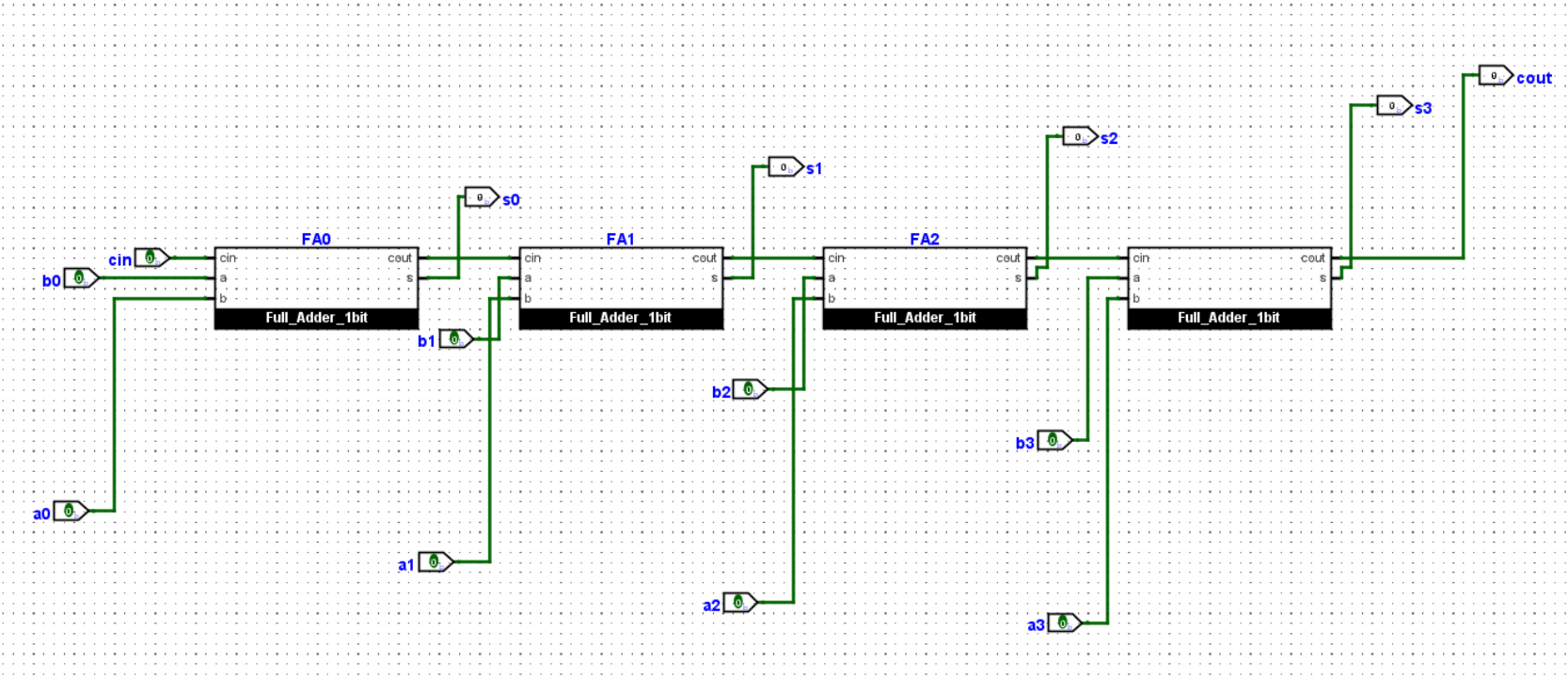




# Part IV Top Level



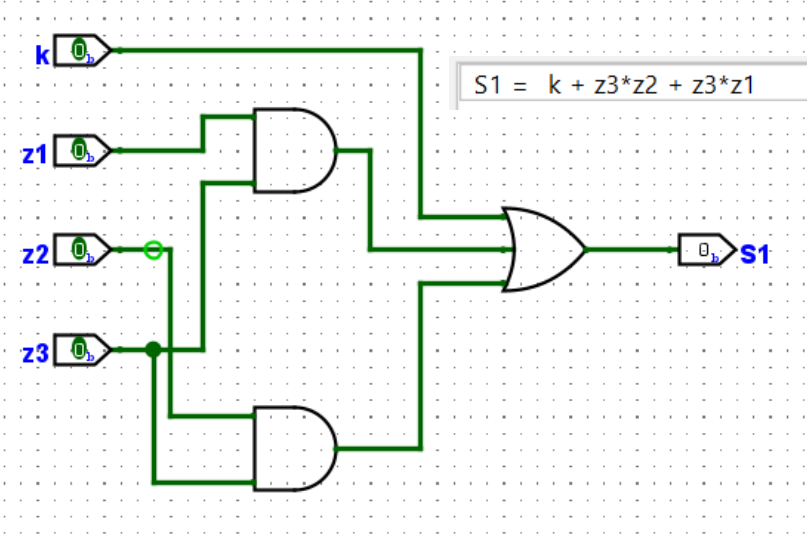
# 4 Bit Full Adder



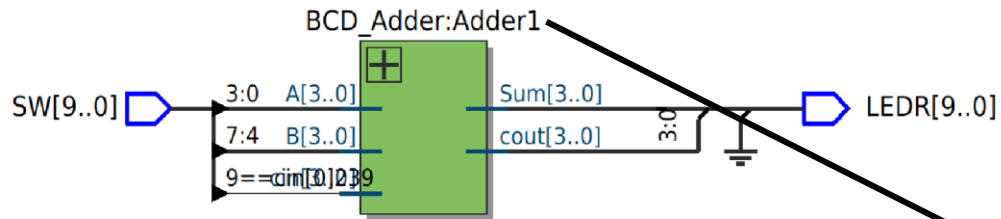
Truth Table of 1 bit Full Adder

| cin | a | b | cout | s |
|-----|---|---|------|---|
| 0   | 0 | 0 | 0    | 0 |
| 0   | 0 | 1 | 0    | 1 |
| 0   | 1 | 0 | 0    | 1 |
| 0   | 1 | 1 | 1    | 0 |
| 1   | 0 | 0 | 0    | 1 |
| 1   | 0 | 1 | 1    | 0 |
| 1   | 1 | 0 | 1    | 0 |
| 1   | 1 | 1 | 1    | 1 |

# BCD Correction



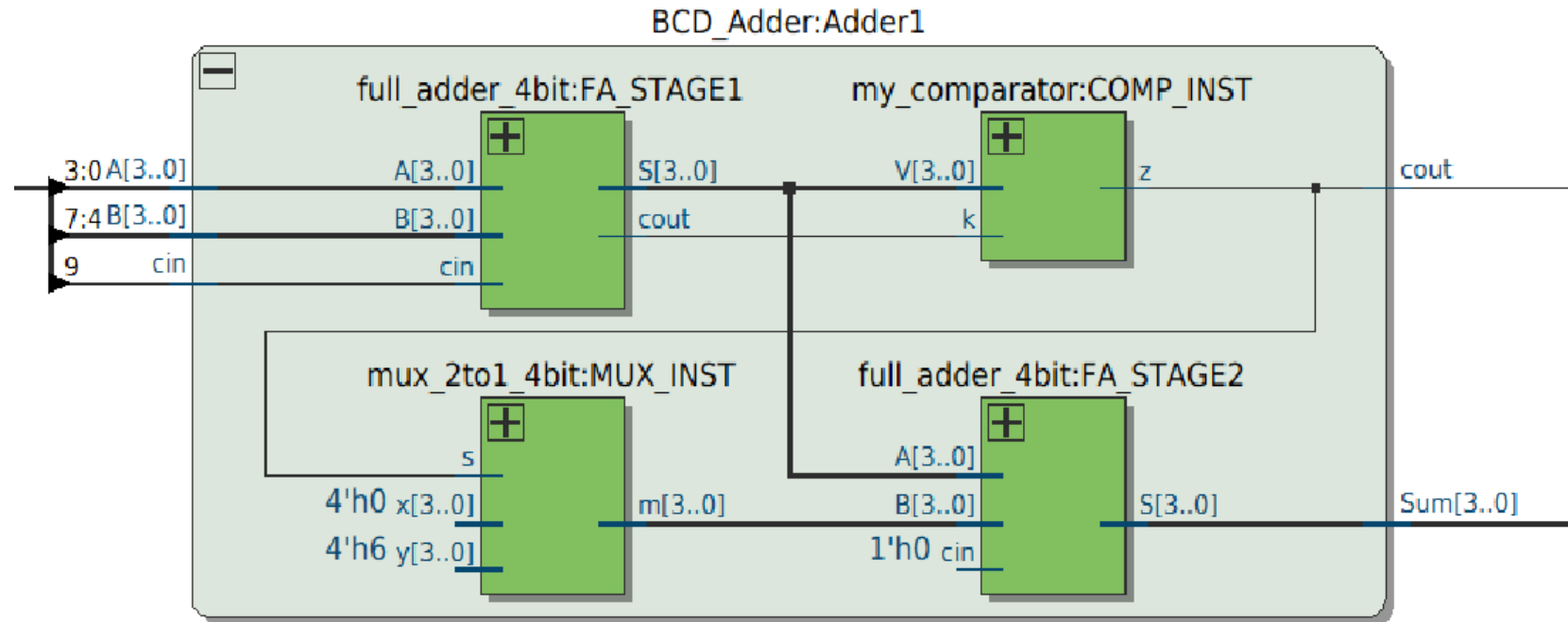
# Main Module



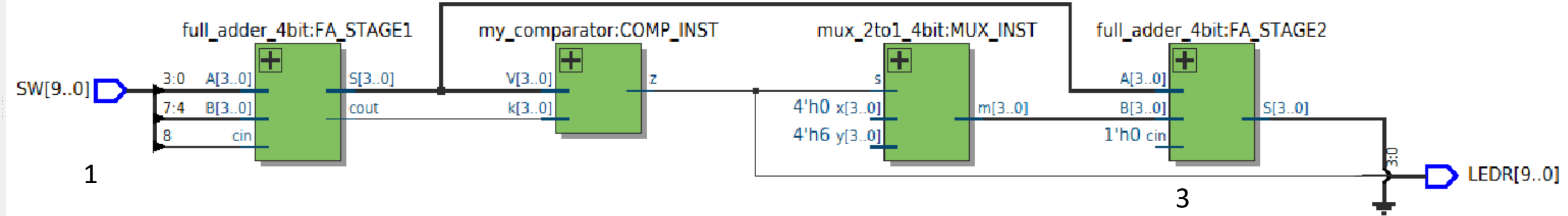
```
module main (  
    input [9:0] SW,  
    output [9:0] LEDR
```

```
);  
BCD_Adder Adder1( .A(SW[3:0]),.B(SW[7:4]),.cin(SW[9]),.Sum(LED[3:0]), .cout(LED[9]));  
endmodule
```

## BCD Adder



# Main Module



```

module main (
    // Pinout assignment
    input wire [9:0] SW,
    output wire [9:0] LEDR
);
    wire [3:0] raw_sum;
    wire k;
    wire comp_z;
    wire adjust_signal;
    wire [3:0] correction_vector;
    wire [3:0] corrected_sum;
    wire dummy_cout;

    // 1. Stage 1 Binary Adder: Calculates A + B + cin
    full_adder_4bit FA_STAGE1 (
        .A(SW[3:0]),
        .B(SW[7:4]),
        .cin(SW[8]),
        .S(raw_sum),
        .cout(k)
    );

    // 2. Comparator Block: Checks if raw_sum > 9
    my_comparator COMP_INST (
        .V(raw_sum), .k(k),
        .z(comp_z)
    );

    // 3. 2-to-1 Multiplexer
    mux_2to1_4bit MUX_INST (
        .s(comp_z),
        .x(4'b0000),
        .y(4'b0110),
        .m(correction_vector)
    );

    // 4. Stage 2 Binary Adder
    full_adder_4bit FA_STAGE2 (
        .A(raw_sum),
        .B(correction_vector),
        .cin(1'b0),
        .S(corrected_sum),
        .cout(dummy_cout)
    );

    // --- LED Output Assignments ---
    assign LEDR[3:0] = corrected_sum;
    assign LEDR[4] = adjust_signal;
    assign LEDR[8:5] = 4'b0000;
    assign LEDR[9] = comp_z;
endmodule

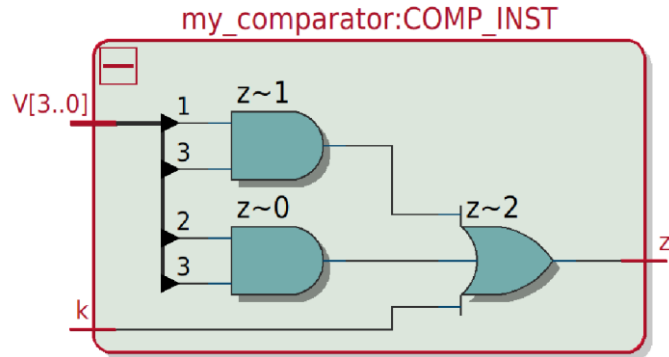
```

# Code for BCD Adder

```
module BCD_Adder (  
    input [3:0] A,  
    input [3:0] B,  
    input cin,  
    output [3:0] Sum,  
    output cout  
);  
    wire [3:0] raw_sum;  
    wire k;  
    wire comp_z;  
    wire [3:0] correction_vector;  
    wire dummy_cout;  
  
    full_adder_4bit FA_STAGE1 (  
        .A(A[3:0]),  
        .B(B[3:0]),  
        .cin(cin),  
        .S(raw_sum),  
        .cout(k)  
    );
```

```
    my_comparator COMP_INST (  
        .V(raw_sum),  
        .k(k),  
        .z(comp_z)  
    );  
  
    assign cout = comp_z;  
  
    mux_2to1_4bit MUX_INST (  
        .s(comp_z),  
        .x(4'b0000),  
        .y(4'b0110),  
        .m(correction_vector)  
    );  
  
    full_adder_4bit FA_STAGE2 (  
        .A(raw_sum),  
        .B(correction_vector),  
        .cin(1'b0),  
        .S(Sum),  
        .cout(dummy_cout)  
    );  
  
endmodule
```

## Comparator



```

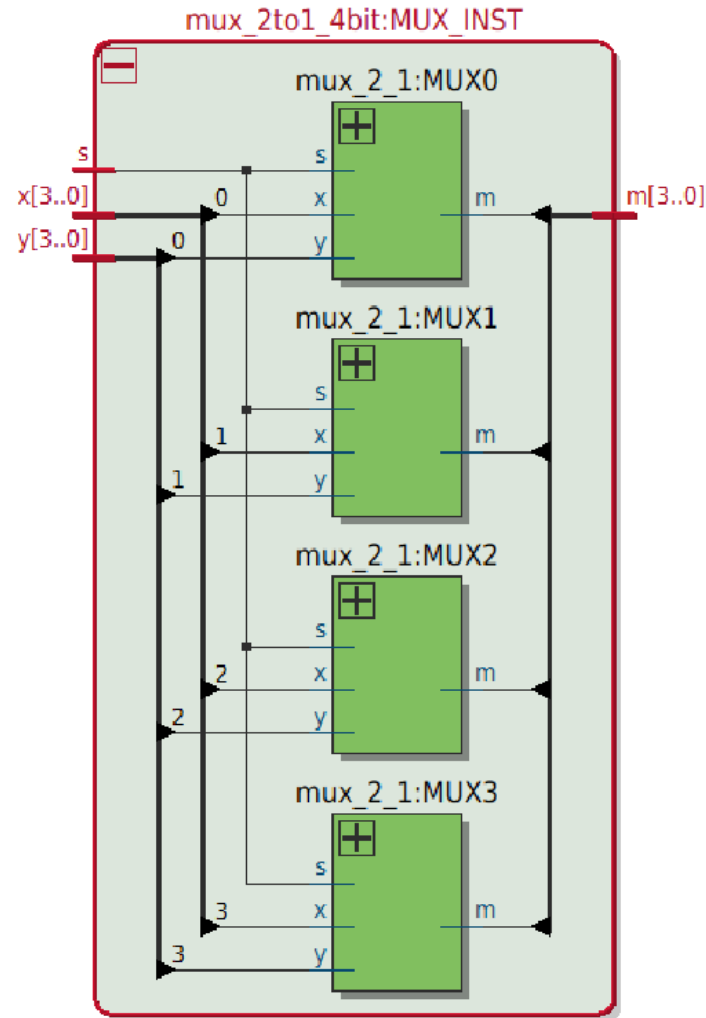
module my_comparator(
  input [3:0] V,
  input  k,
  output  z
);

  // V > 9
  assign z = k | (V[3] & V[2]) | (V[3] & V[1]);

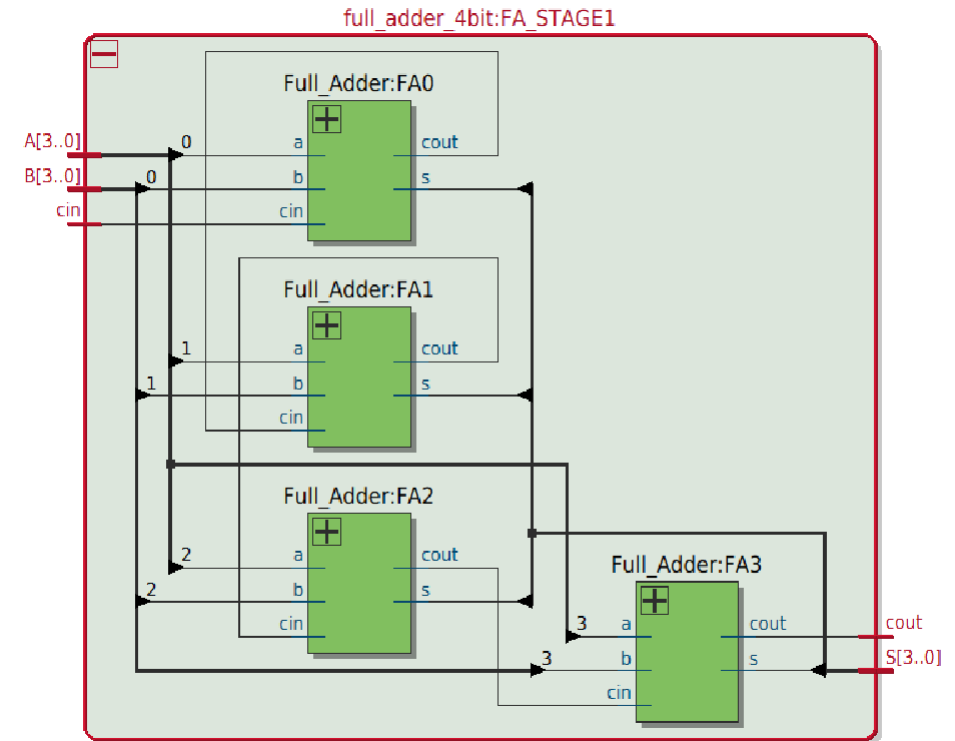
endmodule

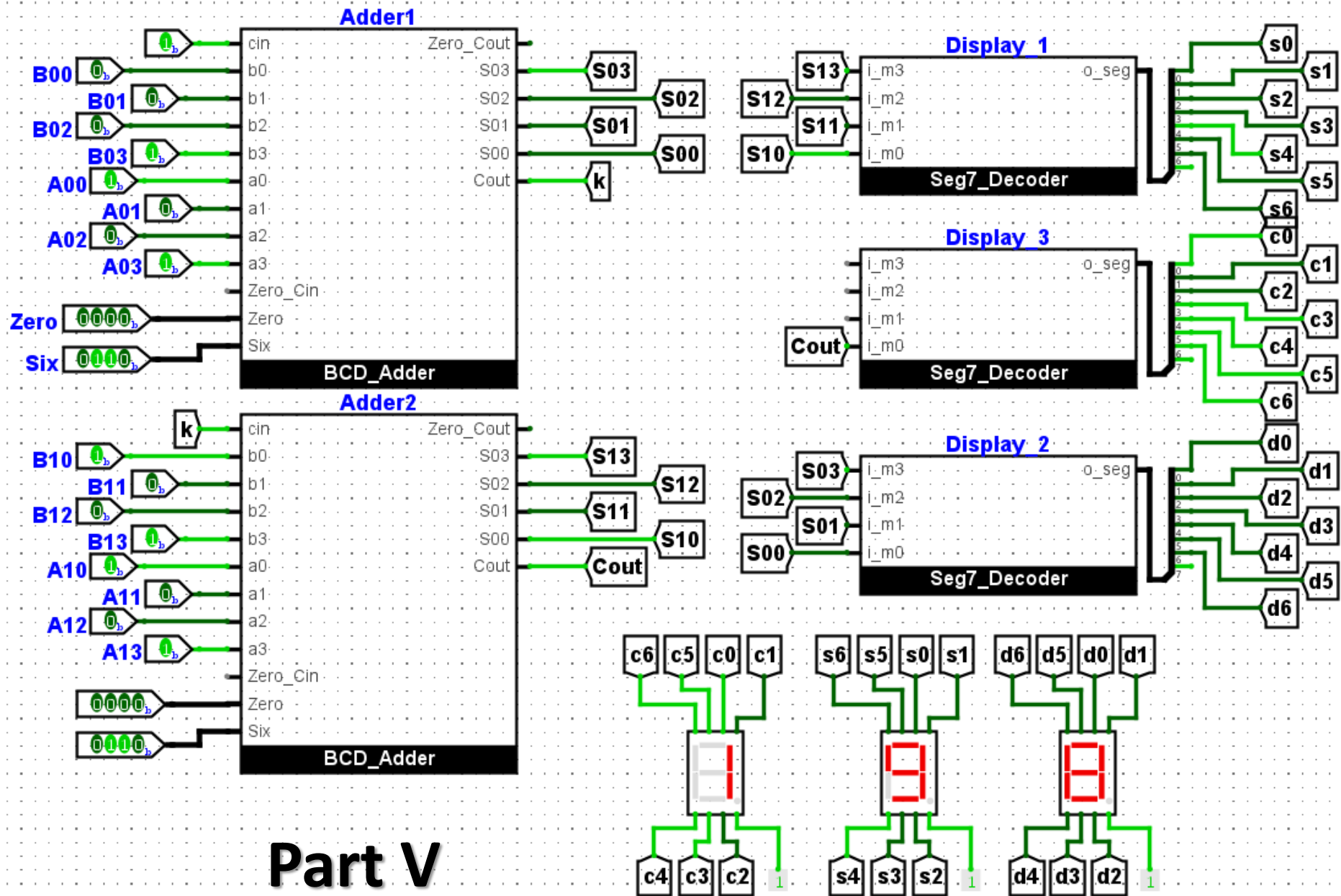
```

## 4 bit MUX

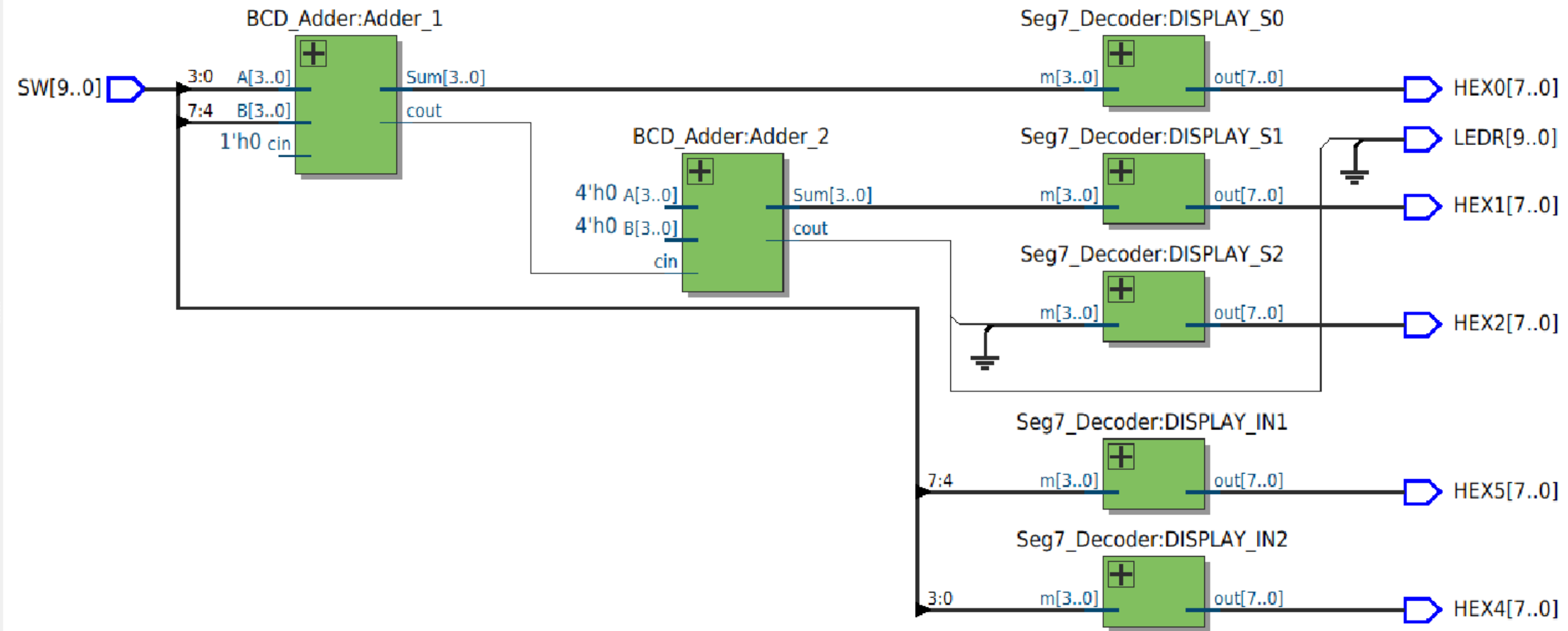


## 4 bit Full Adder





# Main Module



```

module main (
    input [9:0] SW,
        output [9:0] LEDR,
    output [7:0] HEX0,
    output [7:0] HEX1,
    output [7:0] HEX2,
        output [7:0] HEX4,
        output [7:0] HEX5

```

```

);

```

```

    wire [3:0] w0;
    wire [3:0] w1;
    wire [3:0] w2;
    wire      c1;
    wire      c2;

```

```

// Stage 1: Units BCD Adder
BCD_Adder Adder_1 (
    .A(SW[3:0]),
    .B(SW[7:4]),
    .cin(1'b0),
    .Sum(w0),
    .cout(c1)
);

```

## Verilog Code

```

// Stage 2: Tens BCD Adder
BCD_Adder Adder_2 (
    .A(4'b0000),
    .B(4'b0000),
    .cin(c1),
    .Sum(w1),
    .cout(c2)
);

```

```

    assign w2 = {3'b000, c2};
    assign LEDR[9] = c2;
    //Inputs

```

```

    Seg7_Decoder DISPLAY_IN2 (.m(SW[3:0]),
    .out(HEX4));
    Seg7_Decoder DISPLAY_IN1 (.m(SW[7:4]), .out(HEX5));

```

```

    // Seg7 Decoders (Outputs mapped to active-low 7-bit
    displays)

```

```

    Seg7_Decoder DISPLAY_S2 (.m(w2), .out(HEX2));
    Seg7_Decoder DISPLAY_S1 (.m(w1), .out(HEX1));
    Seg7_Decoder DISPLAY_S0 (.m(w0), .out(HEX0));

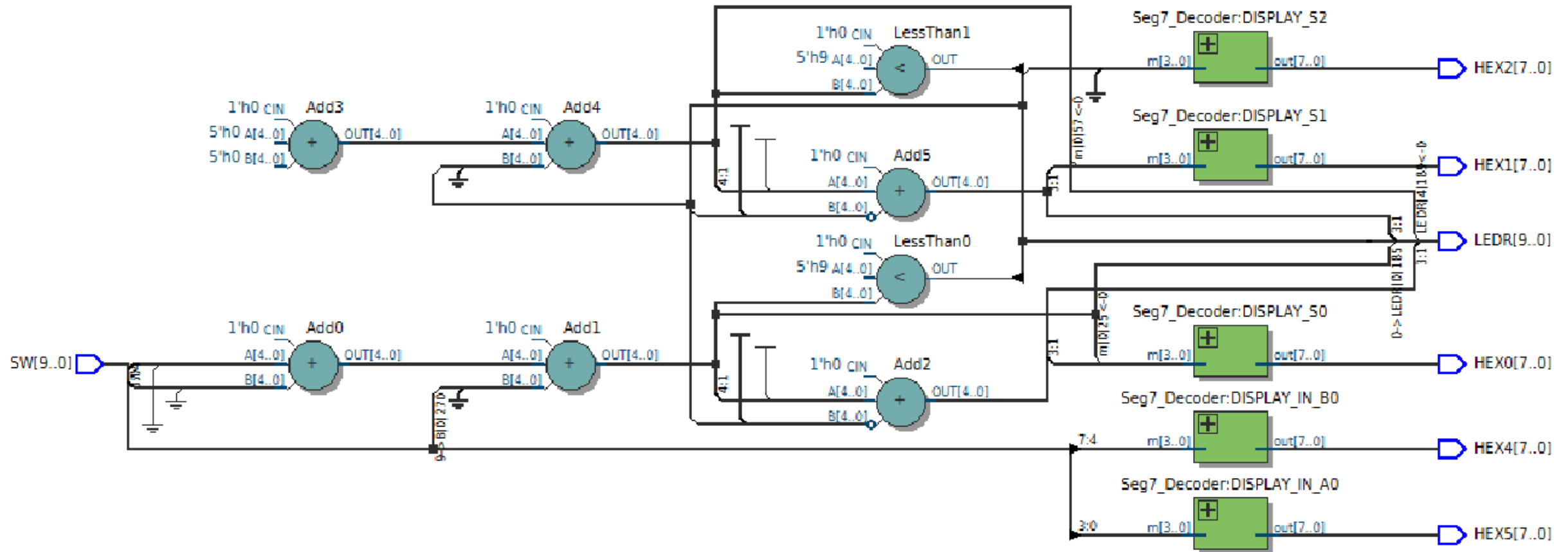
```

```

endmodule

```

## Part VI



# Verilog Code

```
module main (
    input [9:0] SW,
    output [9:0] LEDR,
    output [7:0] HEX0,
    output [7:0] HEX1,
    output [7:0] HEX2,
    output [7:0] HEX4,
    output [7:0] HEX5
);
    wire [3:0] A0 = SW[3:0];
    wire [3:0] B0 = SW[7:4];
    wire [3:0] A1 = 4'b0000;
    wire [3:0] B1 = 4'b0000;
    wire cin;
    assign cin = SW[9];

    reg [4:0] T0;
    reg [4:0] T1;
    reg [3:0] Z0;
    reg [3:0] Z1;
    reg c1;
    reg c2;

    reg [3:0] S0;
    reg [3:0] S1;
    wire [3:0] S2;

    always @(*) begin
        T0 = A0 + B0 + cin;

        if (T0 > 5'd9) begin
            Z0 = 4'd10;
            c1 = 1'b1;
        end else begin
            Z0 = 4'd0;
            c1 = 1'b0;
        end

        S0 = T0 - Z0;

        T1 = A1 + B1 + c1;

        if (T1 > 5'd9) begin
            Z1 = 4'd10;
            c2 = 1'b1;
        end else begin
            Z1 = 4'd0;
            c2 = 1'b0;
        end

        S1 = T1 - Z1;
    end

    assign S2 = {3'b000, c2};

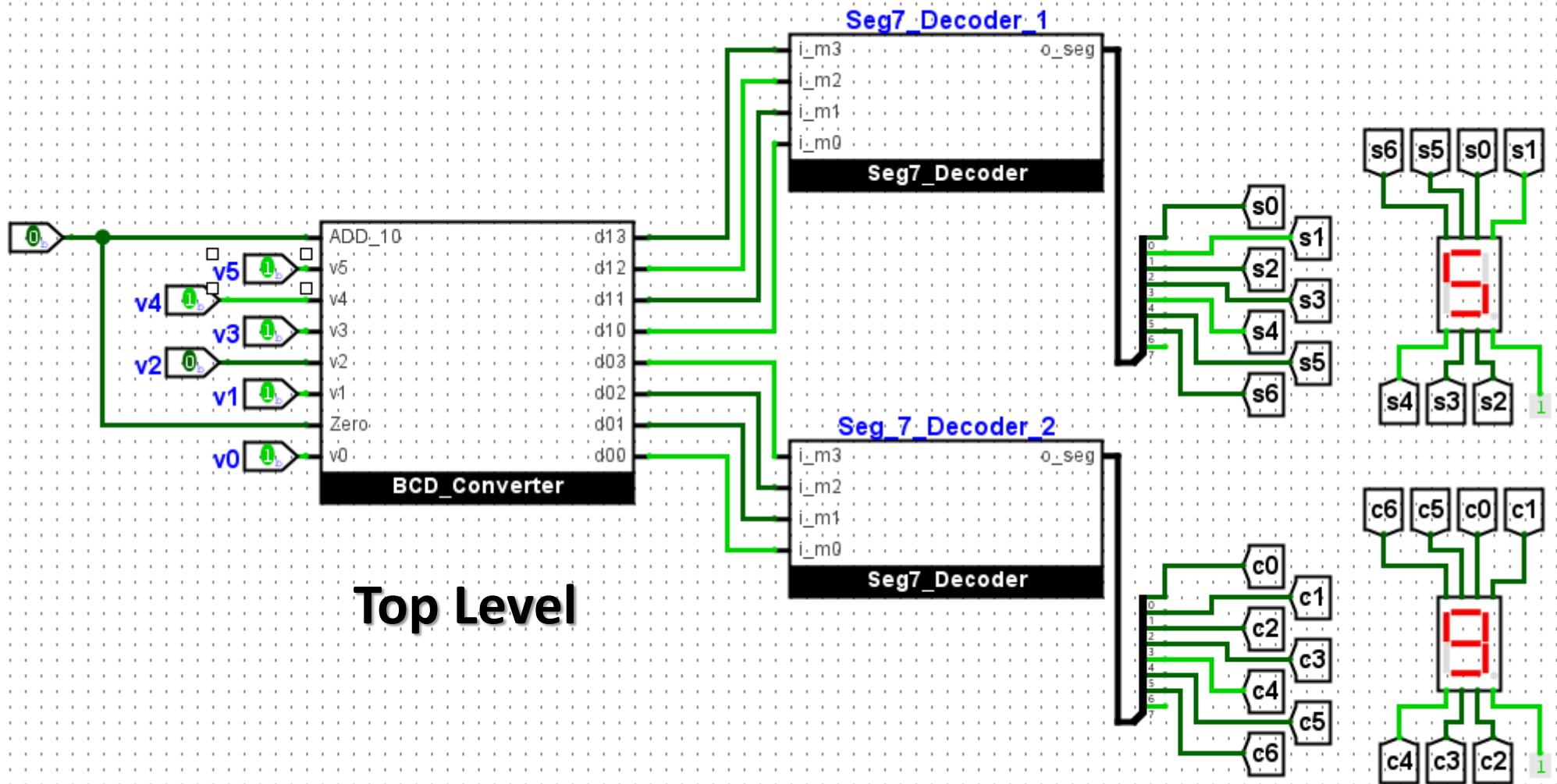
    assign LEDR[3:0] = S0;
    assign LEDR[7:4] = S1;
    assign LEDR[8] = c1;
    assign LEDR[9] = c2;

    Seg7_Decoder
    DISPLAY_IN_A0 (.m(A0),
    .out(HEX5));
    Seg7_Decoder
    DISPLAY_IN_B0 (.m(B0),
    .out(HEX4));

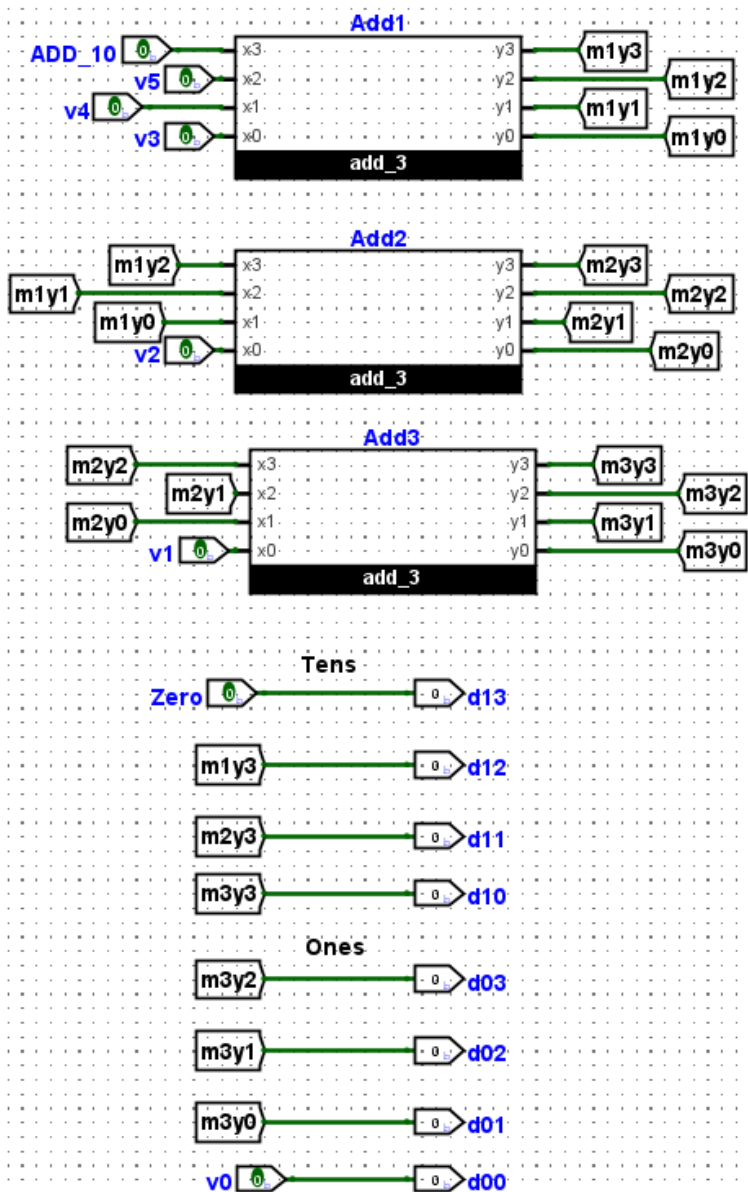
    Seg7_Decoder
    DISPLAY_S2 (.m(S2),
    .out(HEX2));
    Seg7_Decoder
    DISPLAY_S1 (.m(S1),
    .out(HEX1));
    Seg7_Decoder
    DISPLAY_S0 (.m(S0),
    .out(HEX0));

endmodule
```

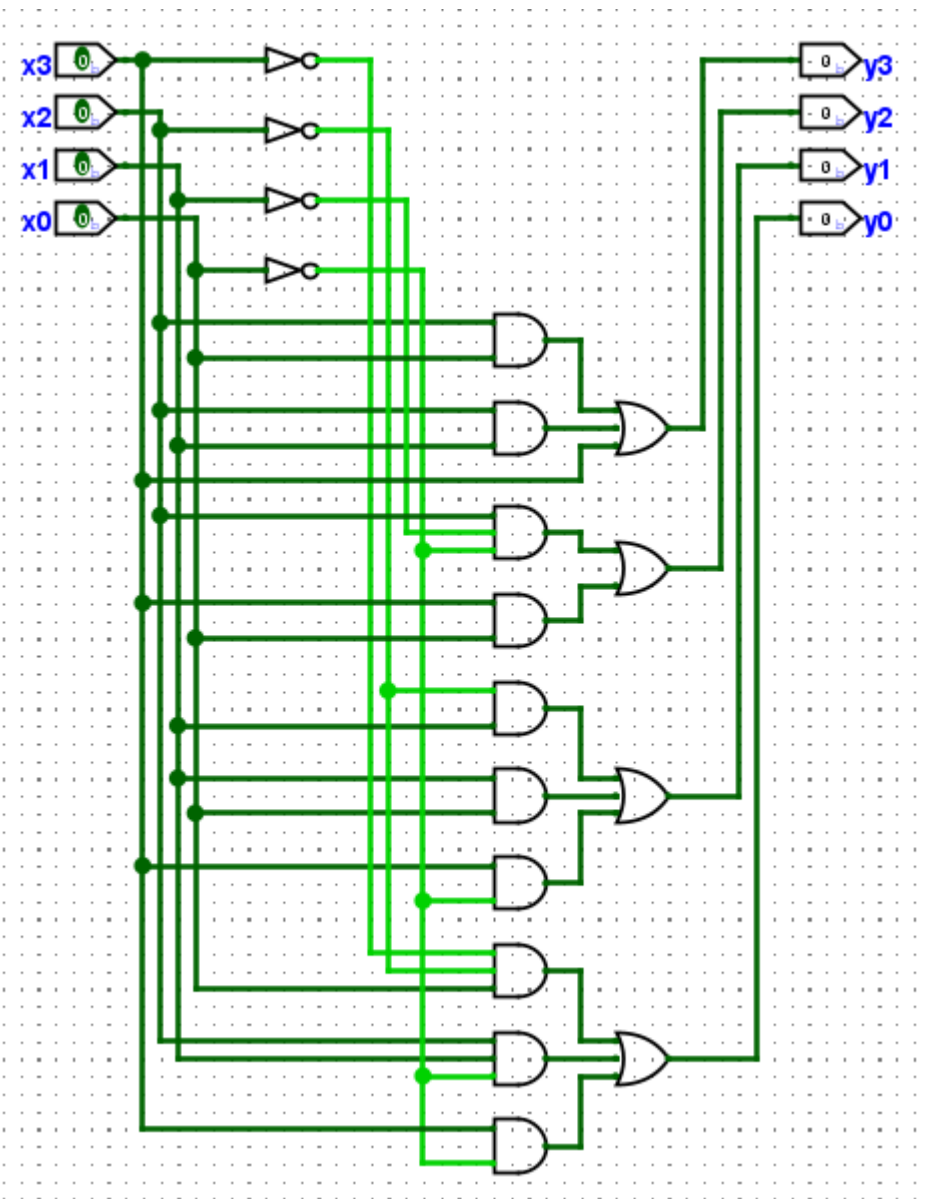
# Part VII



## BCD Converter



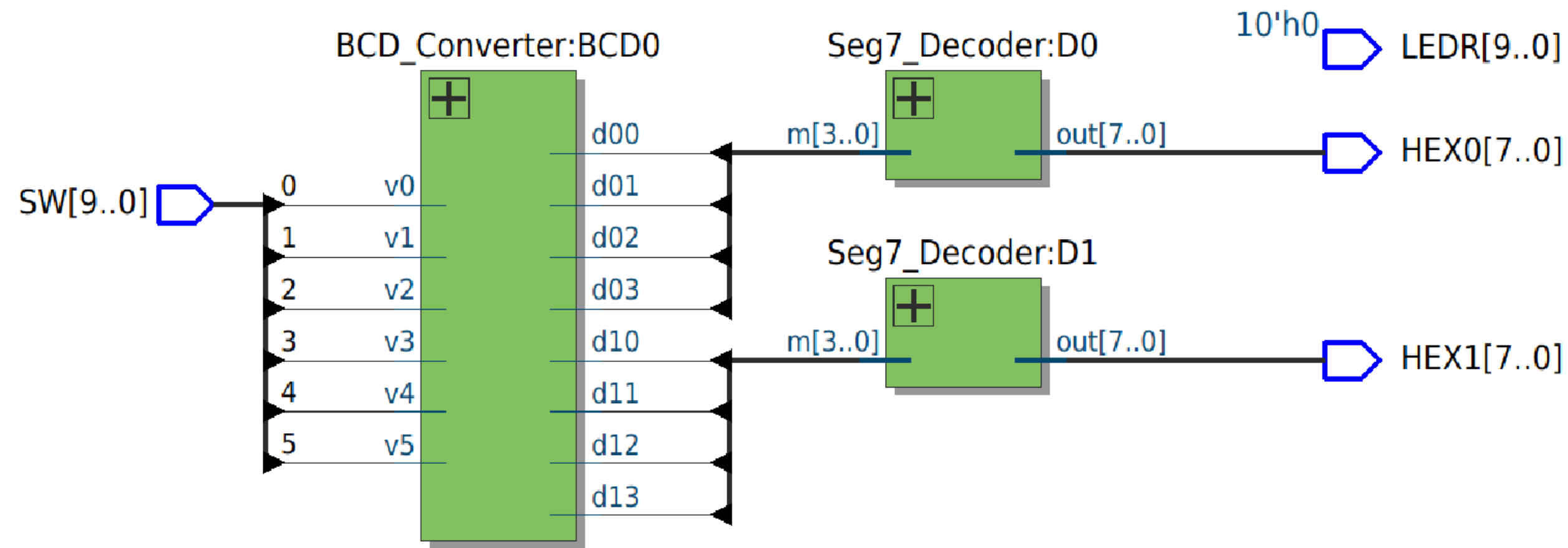
## Add\_3



## Input output relations

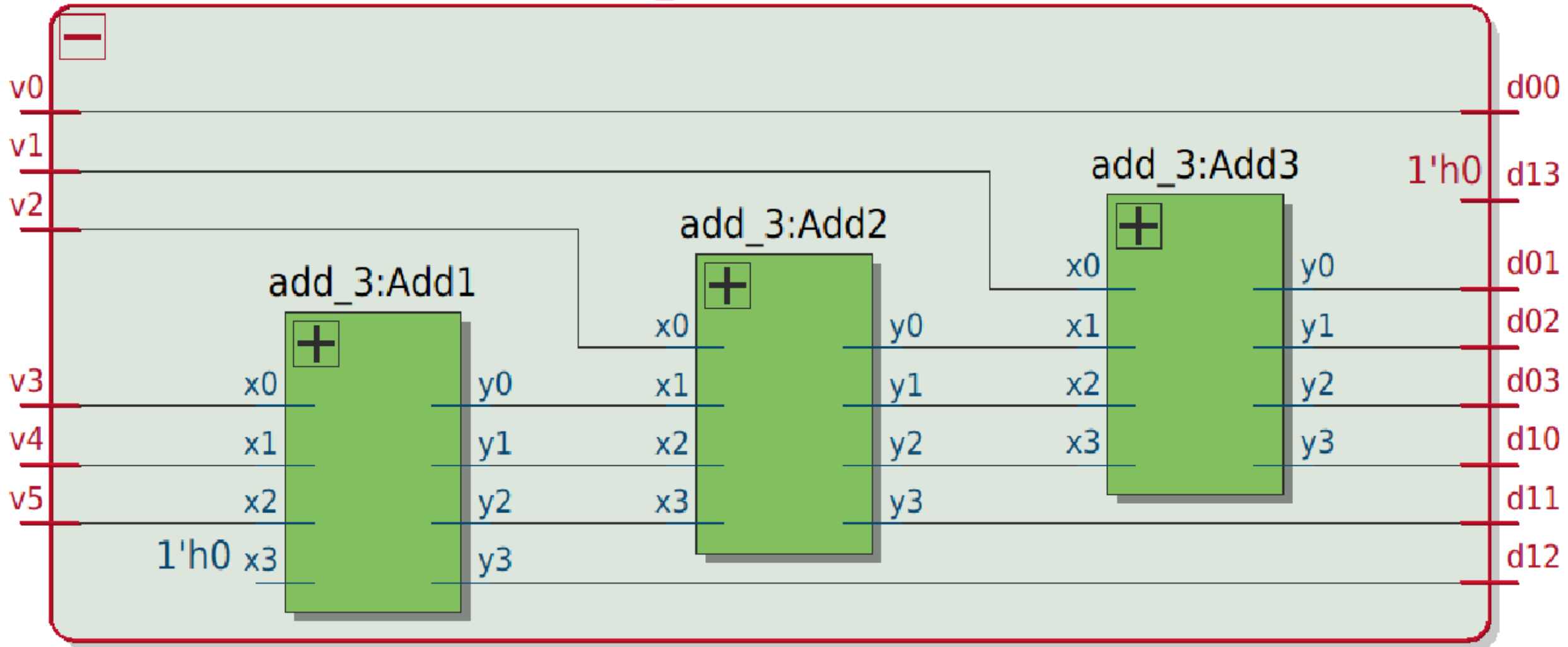
$$\begin{aligned}
 y3 &= x2 \cdot x0 + x2 \cdot x1 + x3 \\
 y2 &= \overline{x2} \cdot \overline{x1} \cdot x0 + x3 \cdot x0 \\
 y1 &= \overline{x2} \cdot x1 + x1 \cdot x0 + x3 \cdot \overline{x0} \\
 y0 &= \overline{x3} \cdot \overline{x2} \cdot x0 + x2 \cdot \overline{x1} \cdot \overline{x0} + x3 \cdot \overline{x0}
 \end{aligned}$$

# Top Module

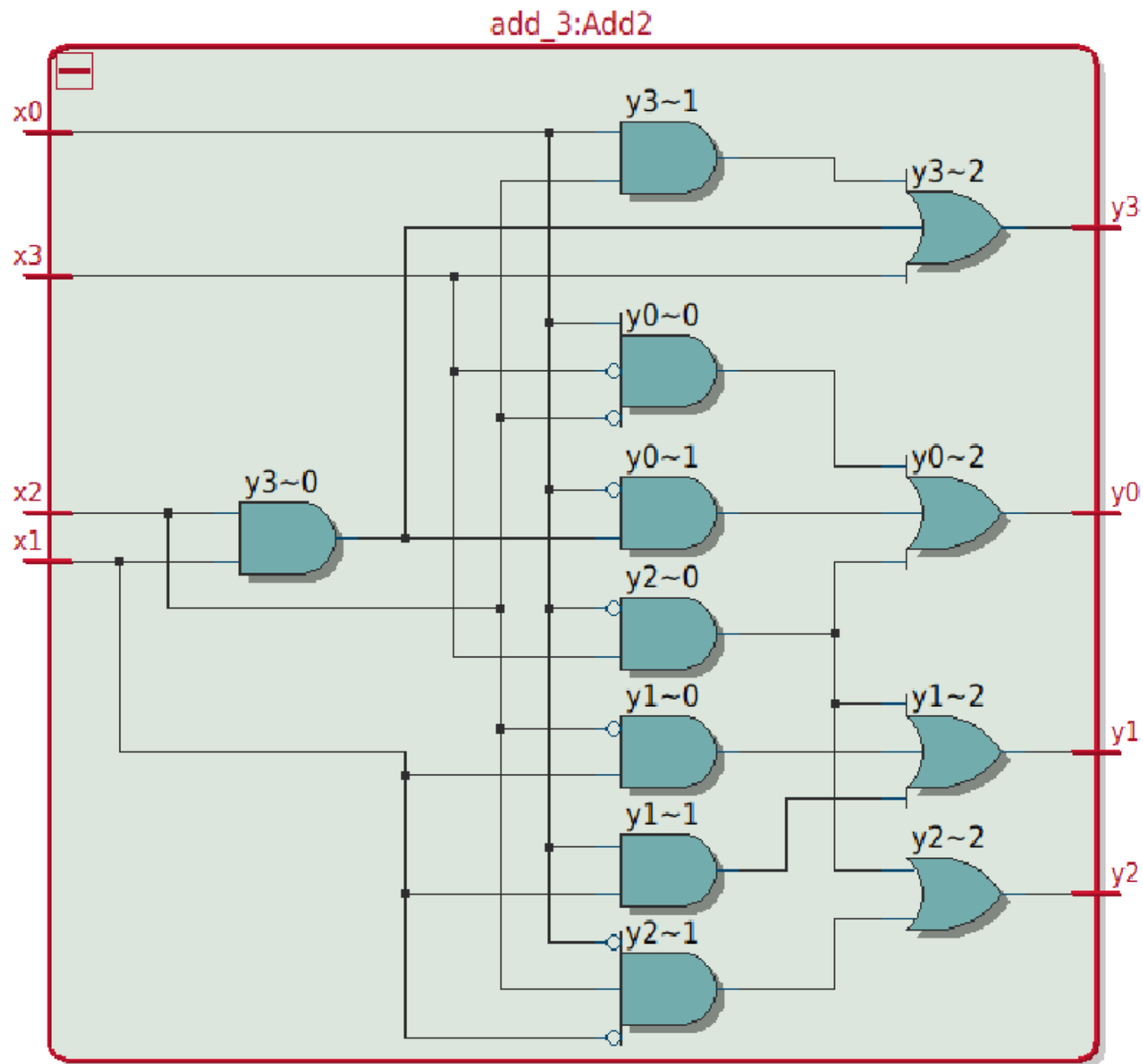


# 6 Digits BCD Converter Module

BCD\_Converter:BCD0



# Add\_3 Module



# Verilog Code – Main

```
1 module main (  
    input [9:0] SW,  
    output [9:0] LEDR,  
    output [7:0] HEX0,  
    output [7:0] HEX1  
);  
  
    // BCD digits  
    wire [3:0] tens;  
    wire [3:0] ones;  
  
    // Ones digit -> HEX0  
    Seg7_Decoder D0 (  
3        .m(ones),  
        .out(HEX0)  
    );  
  
endmodule
```

2

```
// 6-bit Binary to BCD Converter  
BCD_Converter BCD0 (  
    .v5(SW[5]),  
    .v4(SW[4]),  
    .v3(SW[3]),  
    .v2(SW[2]),  
    .v1(SW[1]),  
    .v0(SW[0]),  
    .d13(tens[3]),  
    .d12(tens[2]),  
    .d11(tens[1]),  
    .d10(tens[0]),  
    .d03(ones[3]),  
    .d02(ones[2]),  
    .d01(ones[1]),  
    .d00(ones[0])  
);  
// Tens digit -> HEX1  
Seg7_Decoder D1 (  
    .m(tens),  
    .out(HEX1)  
);
```

# Module Add\_3

```
module add_3(  
    input x3,  
    input x2,  
    input x1,  
    input x0,  
    output y3,  
    output y2,  
    output y1,  
    output y0  
);  
    assign y3 = x3 | (x2 & x1) | (x2 & x0);  
    assign y2 = (x3 & ~x0) | (x2 & ~x1 &  
~x0);  
    assign y1 = (~x2 & x1) | (x1 & x0) | (x3  
& ~x0);  
    assign y0 = (~x3 & ~x2 & x0) |  
        (x2 & x1 & ~x0) |  
        (x3 & ~x0);  
  
endmodule
```

## Module – BCD Converter 6 digits

```
module BCD_Converter(
    input v5,
    input v4,
    input v3,
    input v2,
    input v1,
    input v0,
    output d13,
    output d12,
    output d11,
    output d10,
    output d03,
    output d02,
    output d01,
    output d00
);
    wire m1y3, m1y2, m1y1, m1y0;
    wire m2y3, m2y2, m2y1, m2y0;
    wire m3y3, m3y2, m3y1, m3y0;

    // Add1
    add_3 Add1 (
        .x3(1'b0),
        .x2(v5),
        .x1(v4),
        .x0(v3),
        .y3(m1y3),
        .y2(m1y2),
        .y1(m1y1),
        .y0(m1y0)
    );

    // Add2
    add_3 Add2 (
        .x3(m1y2),
        .x2(m1y1),
        .x1(m1y0),
        .x0(v2),
        .y3(m2y3),
        .y2(m2y2),
        .y1(m2y1),
        .y0(m2y0)
    );

    // Add3
    add_3 Add3 (
        .x3(m2y2),
        .x2(m2y1),
        .x1(m2y0),
        .x0(v1),
        .y3(m3y3),
        .y2(m3y2),
        .y1(m3y1),
        .y0(m3y0)
    );

    // Tens BCD digit
    assign d13 = 1'b0;
    assign d12 = m1y3;
    assign d11 = m2y3;
    assign d10 = m3y3;
    // Ones BCD digit
    assign d03 = m3y2;
    assign d02 = m3y1;
    assign d01 = m3y0;
    assign d00 = v0;
endmodule
```